Software Development Life Cycle (SDLC) Selection for the Project.

1. **Objective(s)**
   - To understand different SDLC models.
   - To learn how to select a suitable SDLC model based on project requirements.
   - To analyze factors affecting SDLC selection.
   - To justify the chosen SDLC model for a given problem.

2. **Problem analysis**

Select the most appropriate SDLC model for the ATM management.

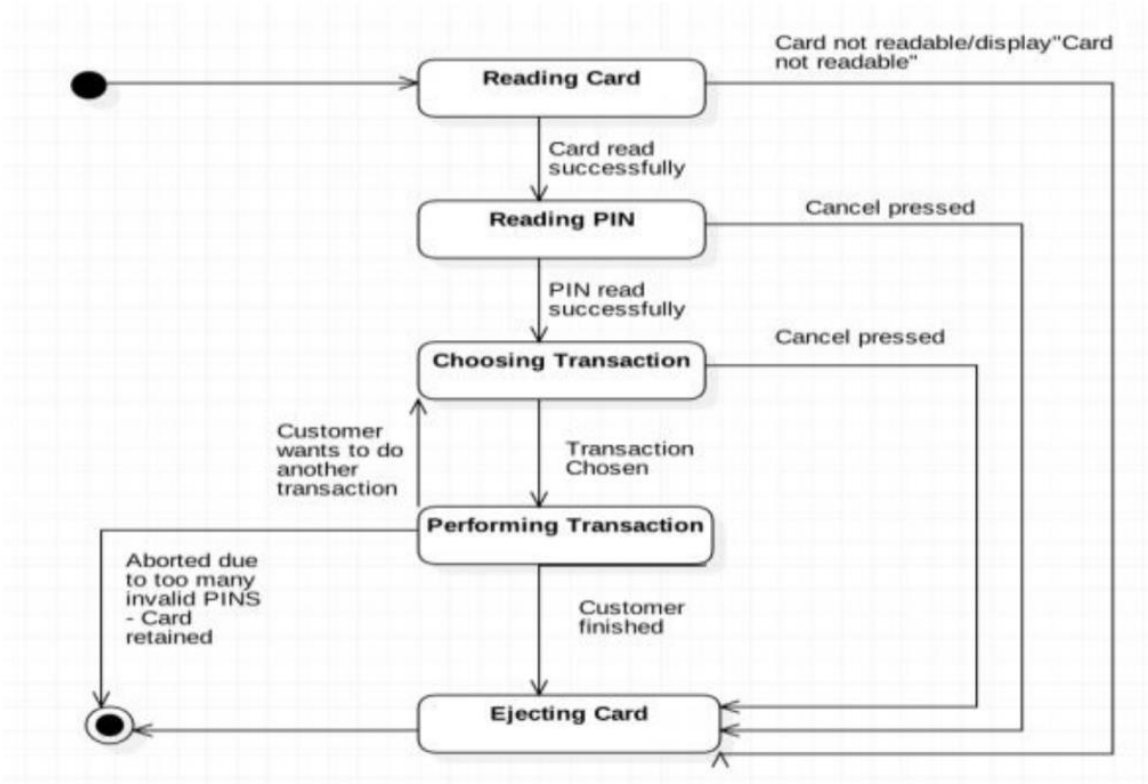

Figure 1: Block diagram of ATM System

3. **SDLC**

Software Development Life Cycle (SDLC) is a structured process used to design, develop, test, and deploy software systems efficiently and with high quality.

The life cycle defines a methodology for improving the quality of software and the overall development process. A typical Software Development Life Cycle consists of the following stages:

<u>Stage 1: Planning and Requirement Analysis</u>

Most fundamental stage in SDLC; performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry. This information is then used to plan the basic project approach and to conduct product feasibility studies in the economical, operational, and technical areas. Planning for the quality assurance requirements and identification of the risks associated with the project is also done in the planning stage.

Stage 2: Defining Requirements

Clearly define and document the product requirements and get them approved by the customer or the market analysts. This is done through an SRS (Software Requirement Specification) document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the Product Architecture

The SRS serves as a reference for product architects to develop the best architecture for the product. Based on the requirements specified in the SRS, more than one design approach is usually proposed and documented in a DDS (Design Document Specification).

This DDS is reviewed by all important stakeholders, and based on various parameters such as risk assessment, product robustness, design modularity, budget, and time constraints; the best design approach is selected. A design approach clearly defines all the architectural modules of the product along with its communication and data flow representation with the external and third-party modules (if any). The internal design of all the modules of the proposed architecture should be clearly defined in detail in DDS.

Stage 4: Building or Developing the Product

In this stage of SDLC, the actual development starts, and the product is built. The programming code is generated as per DDS. If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.

Developers must follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers, etc. are used to generate the code. The programming language is chosen for the type of software being developed.

Stage 5: Testing the Product

This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC. However, this stage refers to the testing-only stage of the product where product defects are reported, tracked, fixed, and retested until the product reaches the quality standards defined in the SRS.

Stage 6: Deployment in the Market and Maintenance

Once the product has been tested and is ready for deployment, it is formally released into the appropriate market. Sometimes, deployment happens in stages according to the organization's business strategy. The product may first be released to a limited segment and tested in a real business environment through **UAT (User Acceptance Testing)**.

Based on the feedback received, the product may then be released as it is or with suggested enhancements to the target market segment. After the product is released, maintenance is carried out for the existing customer base.

3.1 Common SDLC Models

- **Waterfall Model**
  - Sequential approach
  - Suitable for small projects with fixed requirements
- **V-Model**
  - Extension of Waterfall with testing at each stage
  - Suitable for critical systems
- **Iterative Model**
  - Development in repeated cycles
  - Suitable when requirements evolve
- **Incremental Model**
  - System built in parts (increments)
  - Early delivery of partial product
- **Spiral Model**
  - Combines iteration with risk analysis
  - Suitable for large, high-risk projects
- **Agile Model**
  - Flexible and adaptive
  - Continuous feedback and rapid delivery

3.2 Factors for SDLC Selection:

When selecting an SDLC model, consider:

- ➔ Project size (small / medium / large)
- ➔ Requirement clarity (clear / changing)
- ➔ Risk level (low / high)
- ➔ Time constraints
- ➔ Budget
- ➔ Customer involvement
- ➔ Team expertise

4. Methodology

Step 1: Analyze the Problem

- Identify system requirements
- Determine project scope
- Understand user needs

Step 2: Evaluate Project Characteristics

- Requirement stability
- Risk level
- Complexity
- Delivery timeline

Step 3: Compare SDLC Models

- Create a comparison table

Step 4: Select Suitable Model

- Choose the most appropriate model based on analysis.

Step 5: Justify Selection

- Explain why the selected model fits the project better than others.

5. Example of Manual 1

For the ATM Management System, a comparative analysis of different SDLC models was conducted based on selected project criteria. Each criterion was assigned a priority according to its importance to the system. The models were then evaluated against these criteria, and scores were calculated accordingly. It is given in Table 1.

Table 1: Comparison matrix with different models

Priority	Criteria	Waterfall	V-shape	Iterative	Spiral	Agile	Prototype
5	Well known requirement	Yes	Yes	No	No	No	No
3	Technological knowledge	Yes	Yes	No	No	No	No
6	Efficiency	No	Yes	Yes	Yes	No	Yes
3	Risk analysis	No	No	No	Yes	No	No
5	User testing ability	No	No	Yes	Yes	Yes	Yes
5	Dependability and Security	No	Yes	No	Yes	No	No
3	Time consuming	Yes	Yes	No	Yes	No	No
Total- 30	Over all	11	23	11	22	5	11

Justification: The results are as follows: Waterfall (11), V-Shaped (23), Iterative (11), Spiral (22), Agile (5), and Prototype (11).

The V-Shaped model achieved the highest score, indicating strong support for critical project requirements such as well-defined requirements, system reliability, security, and structured testing.

Given that an ATM system requires high dependability, accuracy, and rigorous validation at each stage, the V-Shaped model is the most suitable choice. Its parallel testing approach ensures early detection of errors and enhances overall system quality.

Tasks:

- Select the best model of your system by creating a model matrix.